



Spring Security

Présentation

1

Spring Security

2

- ✓ Introduction
- ✓ Mise en place
- ✓ Filtrage par URL
- ✓ Roles
- ✓ Login / Logout
- ✓ Tag lib JSP
- ✓ Annotations pour le code Java

Spring Security

- Spring Security propose un ensemble de solutions pour les problématiques de sécurité
- Spring Security est la version 2 de Acegi Security
- Spring Acegi
 - propose des utilitaires pour s'interfacer avec des solutions de sécurité
 - ❑ LDAP, SSO CAS...
 - Préconise une approche déclarative
 - ❑ La sécurité devient transverse, gérée via de l'AOP

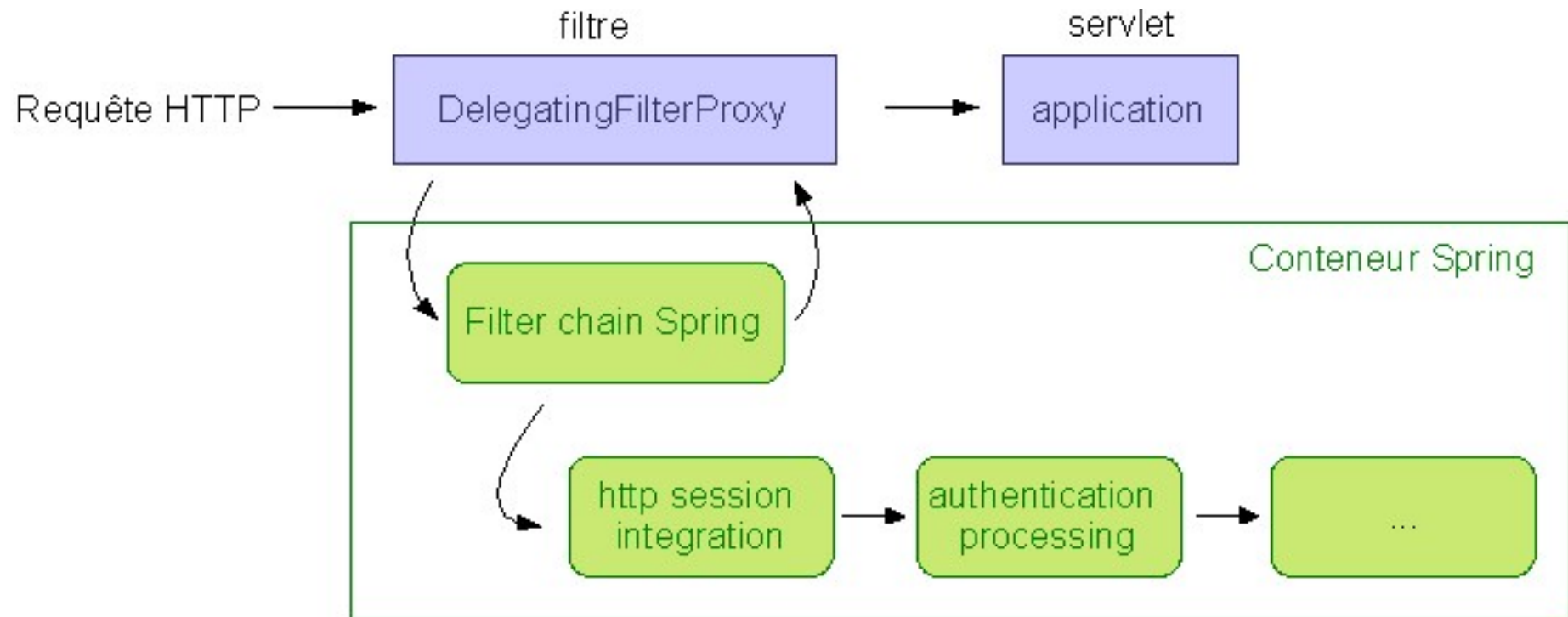
Spring Security

- L'approche déclarative de Spring Security est très puissante
 - Oblige à gérer la sécurité de façon transverse
 - ❑ Le code n'est plus pollué par les préoccupations de sécurité
 - Permet de changer « facilement » de système de sécurité
- Spring Security nécessite une phase d'apprentissage importante
 - PDF de référence fait ~500 pages

Spring Security

- Les requêtes HTTP sont interceptées par un filtre de servlet qui délègue à un bean Spring les traitements de vérification d'accès aux pages web.
- Ce bean met en œuvre une chaîne de filtres. Chacun des filtres est un bean auquel est attribué une tâche précise :
 - Intégration dans la session HTTP des informations de sécurité contenues dans la requête
 - Vérification de l'identité de l'appelant et affichage d'une invite de connexion si nécessaire
 - Vérification des droits d'accès à la ressource sollicitée
 - ...

Spring Security – Architecture



Spring Security

- Certains filtres sont obligatoires, d'autres optionnels.
- La chaîne de filtres est largement configurable, ce qui permet de personnaliser au mieux la gestion de la sécurité dans les applications web.
- Spring Security offre ainsi les fonctionnalités suivantes (en fonction des versions) :
 - Authentification anonyme
 - Fonction Remember Me
 - Gestion NTLM
 - Intégration avec un serveur LDAP ou un serveur CAS
 - Gestion des certificats X509
 - Multi session / mono session
 - Cross-Site Request Forgery
 - ...

Spring Security – Mise en place XML



- Nous allons nous poser dans un contexte JEE
- En configuration XML, deux choses à faire pour mettre en place Spring Security
 - Ajouter un listener dans son fichier web.xml
 - Réaliser le fichier de configuration Spring dédié à Spring security
- Remarque : Spring security n'a aucun rapport avec Spring MVC

Spring Security – Mise en place XML



- Listener à déclarer dans le fichier WEB-INF/web.xml

```
12  <!-- Filtre de Spring Security -->
13  <filter>
14    <filter-name>springSecurityFilterChain</filter-name>
15    <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
16  </filter>
17
18  <filter-mapping>
19    <filter-name>springSecurityFilterChain</filter-name>
20    <url-pattern>/*</url-pattern>
21  </filter-mapping>
```

- Il vous appartient de cibler les URLs

- ☐ Ici /* ⇔ tous les URLs y compris les ressources statiques

- Attention : n'oubliez pas d'ajouter le ContextLoaderListener et le context-param afin que vos fichiers Spring soient chargés dans le contexte JEE
 - Vue dans la partie Spring MVC



Spring Security – Mise en place XML

➤ Fichier de configuration Spring security *basique*

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans:beans xmlns="http://www.springframework.org/schema/security"
3   xmlns:beans="http://www.springframework.org/schema/beans"
4   xmlns:context="http://www.springframework.org/schema/context" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5   xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
6     http://www.springframework.org/schema/security http://www.springframework.org/schema/security/spring-security.xsd
7     http://www.springframework.org/schema/context http://www.springframework.org/schema/context/spring-context.xsd">
8
9   <authentication-manager>
10     <authentication-provider>
11       <user-service>
12         <user name="admin" password="{noop}admin" authorities="ROLE_ADMIN,ROLE_USER" />
13         <user name="guest" password="{noop}guest" authorities="ROLE_USER" />
14       </user-service>
15     </authentication-provider>
16   </authentication-manager>
17
18   <http use-expressions="true">
19     <intercept-url pattern="/user/pageA.jsp" access="hasRole('ROLE_USER')" />
20     <intercept-url pattern="/adm/pageB.jsp" access="hasRole('ROLE_ADMIN')" />
21
22     <http-basic />
23   </http>
24
25   <!-- Obligatoire en Spring Security 6+, mais pas dans les versions anterieures -->
26   <beans:bean
27     class="org.springframework.web.servlet.handler.HandlerMappingIntrospector"
28     id="mvcHandlerMappingIntrospector" />
29
30 </beans:beans>
```

Spring Security – Mise en place XML



➤ **<authentication-manager>**

➤ Vous permet de déclarer des **<authentication-provider>**

➤ **<authentication-provider>** définit les objets qui vont fournir les informations d'authentications

➤ Dans notre exemple on fait usage d'une liste écrite en dur dans le fichier XML : **<user-service>**

➤ On peut aussi

☐ Se brancher sur un LDAP ou une base de données

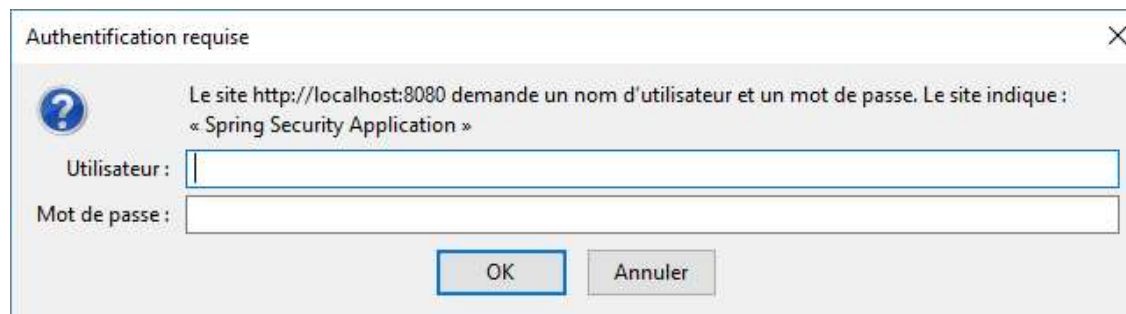
☐ Créer sa propre classe

Spring Security – Mise en place XML



➤ <http>

- intercept-url : Permet de dire le/les ressources à sécuriser en fonction d'un ou plusieurs rôles
- <http-basic /> : indique le moyen mis en place pour l'authentification web
- ❑ Ici la page spécifique du navigateur (ici Firefox)



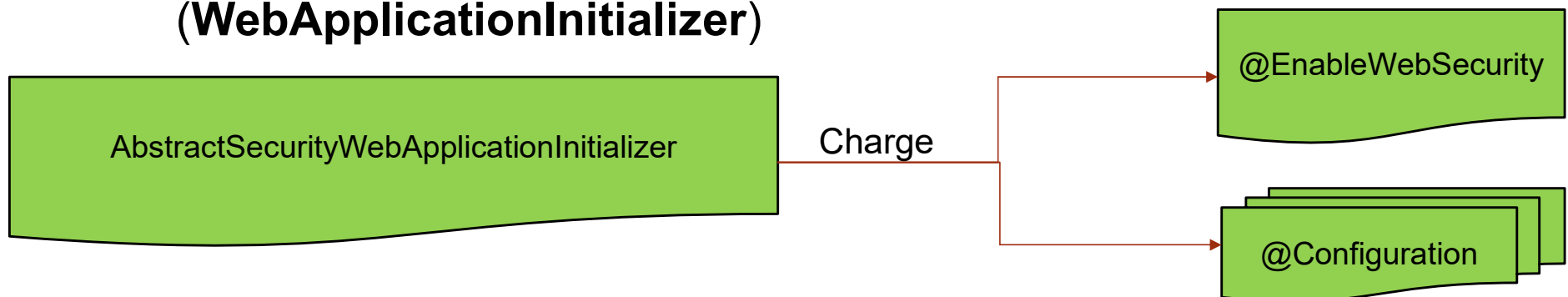
Spring Security – Mise en place

- Par défaut, vous aurez les pages du serveur JEE
 - Si l'utilisateur n'a pas le bon rôle : 403
 - Si l'utilisateur se trompe lors de l'authentification : 401
- Vous pouvez changer ce comportement et même les codes d'erreurs si vous le souhaitez
 - Via votre serveur (apache ou JEE)
 - Via le framework Spring

Spring Security – Mise en place @



- Il n'y a rien à placer dans le fichier web.xml
- Si vous préférez faire usage de la configuration par annotation il y a deux choses à faire pour mettre en place Spring Security
 - Créer une classe de configuration dédiée au Spring Security (**@EnableWebSecurity**)
 - Créer une classe qui portera la configuration JEE (**WebApplicationInitializer**)



Spring Security – Mise en place @



- La classe de configuration dédiée au Spring Security
 - En plus de @Configuration elle portera l'annotation **@EnableWebSecurity**
 - Elle héritera d'une classe qui implémente l'interface **SecurityConfigurer**
 - ❑ Dans ce cas précis elle héritera de **WebSecurityConfigurerAdapter**

Ancienne
approche

```
@Configuration
@EnableWebSecurity
public class MaConfigurationSpringSecurity extends WebSecurityConfigurerAdapter {

    @Autowired
    public void configureGlobalSecurity(AuthenticationManagerBuilder auth) throws Exception {
        // Ne pas indiquer ROLE_, c'est ajouté automatiquement
        final InMemoryUserDetailsManagerConfigurer<AuthenticationManagerBuilder> memory = auth.inMemoryAuthentication();
        memory.withUser("admin").password("admin").roles("ADMIN", "USER");
        memory.withUser("guest").password("guest").roles("USER");
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.authorizeRequests().antMatchers("/").permitAll();
        http.authorizeRequests().antMatchers("/user/pageA.jsp").access("hasRole('ROLE_USER')");
        http.authorizeRequests().antMatchers("/adm/pageB.jsp").access("hasRole('ROLE_ADMIN')");
        // Pour tous les autres URL : il faut être authentifié
        http.authorizeRequests().anyRequest().authenticated();
        // Authentification en mode basic
        http.authorizeRequests().and().httpBasic();
    }
}
```

Spring Security – Mise en place @



- Si votre version de Spring le permet, au lieu de créer une méthode ***configureGlobalSecurity*** vous pouvez surcharger la méthode ***configure(AuthenticationManagerBuilder)***

Ancienne
approche

```
@Configuration
@EnableWebSecurity
public class MaConfigurationSpringSecurity extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws Exception {
        // Ne pas indiquer ROLE_, c'est ajouté automatiquement
        final InMemoryUserDetailsManagerConfigurer<AuthenticationManagerBuilder> memory = auth.inMemoryAuthentication();
        memory.withUser("admin").password("admin").roles("ADMIN", "USER");
        // Un autre
        memory.withUser("guest").password("guest").roles("USER");
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.authorizeRequests().antMatchers("/user/pageA.jsp").access("hasRole('ROLE_USER')");
        http.authorizeRequests().antMatchers("/adm/pageB.jsp").access("hasRole('ROLE_ADMIN')");
        // Pour tous les autres URL : il faut être authentifié
        http.authorizeRequests().anyRequest().authenticated();
        // Authentification en mode basic
        http.authorizeRequests().and().httpBasic();
    }
}
```


Spring Security – Mise en place @



- Si votre version de Spring le permet, il n'est plus conseillé d'hériter ou d'implémenter une interface, mais de déclarer des @Bean directement associés à la sécurité

Approche
Spring
Security 5

```
@Configuration
@EnableWebSecurity
public class MaConfigurationSpringSecurity {

    @Bean
    public InMemoryUserDetailsManager userDetailsService() {
        SpringSecurityConfiguration.LOG.debug("SpringSecurityConfiguration - configure(AuthenticationManagerBuilder)");
        // Ne pas indiquer ROLE_, c'est ajoute automatiquement
        var userAdmin = User.withUsername("admin").password("{noop}admin").roles("ADMIN", "USER").build();
        var userGuest = User.withUsername("guest").password("{noop}guest").roles("USER").build();
        return new InMemoryUserDetailsManager(userAdmin, userGuest);
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeRequests().antMatchers("/user/pageA.jsp").access("hasRole('ROLE_USER')");
        http.authorizeRequests().antMatchers("/adm/pageB.jsp").access("hasRole('ROLE_ADMIN')");
        // Pour tous les autres URL : il faut être authentifié
        http.authorizeRequests().anyRequest().authenticated();
        // Authentification en mode basic
        http.authorizeRequests().and().httpBasic();
        return http.build();
    }
}
```

Spring Security – Mise en place @



- A partir de Spring Security 6, les méthodes changes (encore ...)

Approche
Spring
Security 6+

```
@Configuration
@EnableWebSecurity // Si vous n'êtes pas en Spring Boot
public class MaConfigurationSpringSecurity {

    @Bean
    public InMemoryUserDetailsManager userDetailsService() {
        // Ne pas indiquer ROLE_, c'est ajoute automatiquement
        var userAdmin = User.withUsername("admin").password("{noop}admin").roles("ADMIN", "USER").build();
        var userGuest = User.withUsername("guest").password("{noop}guest").roles("USER").build();
        return new InMemoryUserDetailsManager(userAdmin, userGuest);
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(authorize -> authorize
            .requestMatchers(new AntPathRequestMatcher("/"), new AntPathRequestMatcher("/menu.jsp")).permitAll()
            .requestMatchers(new AntPathRequestMatcher("/user/pageA.jsp*")).hasRole("USER")
            .requestMatchers(new AntPathRequestMatcher("/adm/pageB.jsp*")).hasRole("ADMIN").anyRequest()
            .authenticated()).httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

Spring Security – Mise en place @



➤ La classe qui portera la configuration JEE

➤ Elle héritera d'une classe qui implémente l'interface **WebApplicationInitializer**

❑ Dans ce cas précis, elle héritera de **AbstractSecurityWebApplicationInitializer**

❑ Elle s'occupe de monter le filtre et dans le cas où il n'y a pas de Spring MVC, elle charge aussi la configuration du Spring Security

```
public class SpringSecurityInitializer extends AbstractSecurityWebApplicationInitializer {  
  
    public SpringSecurityInitializer() {  
        // Ici, il n'y a pas de Spring MVC  
        super(MaConfigurationSpringSecurity.class);  
    }  
}
```

➤ Dans le cas où vous faites du **Spring MVC** elle ne devra pas faire un `super(xxx.class)`

❑ En effet dans ce cas, c'est la classe **AbstractAnnotationConfigDispatcherServletInitializer** qui devra charger la classe de configuration du Spring Security

Spring Security – Mise en place @

➤ En Spring MVC

```
public class SpringSecurityInitializer extends AbstractSecurityWebApplicationInitializer {  
  
    public SpringSecurityInitializer() {  
        // Ici, il y a du Spring MVC, donc on ne fait RIEN  
        // Mais il est impératif d'écrire la classe sinon Spring Security ne marchera pas  
    }  
}
```

AbstractSecurityWebApplicationInitializer

AbstractAnnotationConfigDispatcherServletInitializer

Charge

@EnableWebSecurity

@EnableWebMvc

@Configuration

Travaux pratiques

- Exo 19 - Mettez en place Spring security dans un projet ultra basique
- Importez le premier exercice puis mettez en place la gestion des droits sur les pageA et pageB
 - pageA : il faut au moins le ROLE_USER
 - pageB : il faut au moins le ROLE_ADMIN
 - menu : accessible à tous



Spring Security – <form-login>

- A la place de <http-basic/> vous pouvez mettre <form-login/>
 - Sans plus d'information, c'est Spring Security qui vous fabriquera une page web pour l'authentification



Login with Username and Password

User:

Password:

Spring Security – <form-login>

- Il est conseillé de réaliser sa propre page de login
 - Entre autres, afin de gérer l'internationalisation
- Il suffit de paramétrer <form-login> via ses nombreux attributs
 - *login-page* : URL de votre page de login
 - *default-target-url* : URL si authentication ok
 - *authentication-failure-url* : URL si authentication ko

Spring Security 3 – <form-login>

- Par défaut, les attributs de votre page login devront porter les valeurs / noms suivants :
 - *action* : de votre formulaire devra être **/j_spring_security_check**
 - ❑ Paramétrable via l'attribut *login-processing-url* de <form-login>
 - *name* : de l'input login devra être **j_username**
 - ❑ Vous pouvez le changer via l'attribut *username-parameter* de <form-login> (qui ne marche pas toujours)
 - *name* : de l'input password devra être **j_password**
 - ❑ Vous pouvez le changer via l'attribut *password-parameter* de <form-login> (qui ne marche pas toujours)
- Remarque : les informations dans l'aide XML du namespace spring security **ne sont pas justes**

- Par défaut, les attributs de votre page login devront porter les valeurs / noms suivants :
 - *action* : de votre formulaire devra être **/login**
 - ❑ Paramétrable via l'attribut *login-processing-url* de <form-login>
 - *name* **ET** *id* : de l'input login devra être **username**
 - ❑ Vous pouvez le changer via l'attribut *username-parameter* de <form-login>
 - *name* **ET** *id* : de l'input password devra être **password**
 - ❑ Vous pouvez le changer via l'attribut *password-parameter* de <form-login>

Spring Security – <form-login>

➤ Exemple de paramétrage code :

```
// Règles des matcher
// ...
// Pour tous les autres URL : il faut être authentifié
http.authorizeRequests().anyRequest().authenticated();

// On utilise notre page de login
FormLoginConfigurer<HttpSecurity> formLogin = http.authorizeRequests().and().formLogin();
// Toutes les pages associées à login sont accessibles sans authentification
formLogin.permitAll();
// Paramétrage pour Login
formLogin.loginPage("/login.jsp");
formLogin.loginProcessingUrl("/login"); // <form action="/login" ...>
formLogin.failureUrl("/login.jsp?error=true"); // Par exemple si on veut revenir sur login
formLogin.defaultSuccessUrl("/menu.jsp", true); // Après authentification on veut passer par menu.jsp
// Si on met false, le comportement est le suivant :
// - je ne suis pas connecté et je demande http://.../listeCompte.smv
// - je suis redirigé vers /login
// - je m'authentifie => ok => va sur la page initialement demandée = /listeCompte.smv
```

Spring Security – <form-login>

➤ Exemple de paramétrage code :

Approche
Spring
Security 6+

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

    http.authorizeHttpRequests(...)
        .formLogin(formLogin -> formLogin.loginProcessingUrl("/login")
            .defaultSuccessUrl("/menu.jsp", true)
            .failureUrl("/login.jsp?error=true")
            .loginPage("/login.jsp")
            .permitAll()

        );

    return http.build();
}
```

Spring Security – CSRF

- En Spring Security le **CSRF** (Cross Site Request Forgery) est
 - activé par défaut en Spring 4
 - désactivé par défaut en Spring Security 3

- Quand il est actif, cela implique qu'il est obligatoire de placer dans ses `<form>` un attribut web

```
<input type="hidden" name="${_csrf.parameterName}" value="${_csrf.token}"/>
```

- Pour le désactiver, pensez à ajouter la balise **`<csrf disabled="true"/>`** dans votre `<http>`, ou en code à appeler

```
http.authorizeRequests().and().csrf().disable();
```

```
http.csrf(csrf -> csrf.disable());
```

Spring Security – PermitAll

- Pour rendre une page accessible à tous, il suffit de déclarer un `<http>` avec **`security="none"`**
- Dans le cas du paramétrage code, vous pouvez faire usage de `permitAll()` :

```
http.authorizeRequests().antMatchers("/", "/js/**", "/img/**").permitAll();
```

- On s'en sert souvent sur les pages d'erreurs et/ou les ressources statiques (JS, Images, ...)

```
http.authorizeHttpRequests(authorize -> authorize  
    .requestMatchers(new AntPathRequestMatcher("/"),  
        new AntPathRequestMatcher("/js/**")).permitAll()
```

6.1+

Spring Security – Erreurs et jsessionid

- Vous pouvez récupérer dans vos JSP la dernière erreur de Spring security à travers l'attribut **SPRING_SECURITY_LAST_EXCEPTION**
- Depuis la version 4.2+ de Spring Security, si votre URL contient un ';' il ne sera plus accepté par défaut
 - ➡ Hors le jsessionid est souvent placé dans l'URL après un ';'

```
<!-- Permet d'avoir un URL en /xxx;jsessionid=1499806E2058A78F97B1F55173DB239D -->  
<beans:bean id="httpFirewall" class="org.springframework.security.web.firewall.StrictHttpFirewall">  
  <beans:property name="allowSemicolon" value="true" />  
</beans:bean>  
<http-firewall ref="httpFirewall" />
```

```
@Bean  
public StrictHttpFirewall httpFirewall() {  
    StrictHttpFirewall firewall = new StrictHttpFirewall();  
    firewall.setAllowSemicolon(true);  
    return firewall;  
}
```

Spring Security – <access-denied-handler>

- La balise <access-denied-handler> se déclare dans <http> et permet de rediriger vers un objet (attribut *ref*) ou un URL (attribut *error-page*) en cas de problème **de rôle**

➤ Via le code

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.authorizeRequests().antMatchers("/user/pageA.jsp").access("hasRole('ROLE_USER')");
    // Pour tous les autres URL : il faut être authentifié
    http.authorizeRequests().anyRequest().authenticated();
    // On gère nous même la page de problème associée aux rôles
    http.authorizeRequests().and().exceptionHandling().accessDeniedPage("/mauvaisrole.jsp");
}
```

- Ne pas oublier de donner des droits d'accès à la page qui servira d'erreur
 - none ou autre selon vos contraintes

Spring Security – <access-denied-handler>

32

6.1+

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    SpringSecurityConfiguration.LOG.debug("SpringSecurityConfiguration - filterChain(HttpSecurity)");

    http.csrf(csrf -> csrf.disable());

    http.authorizeHttpRequests(authorize -> authorize.requestMatchers(new AntPathRequestMatcher("/")).permitAll()
        .requestMatchers(new AntPathRequestMatcher("/user/pageA.jsp*")).hasRole("USER")
        .requestMatchers(new AntPathRequestMatcher("/adm/pageB.jsp*")).hasRole("ADMIN").anyRequest()
        .authenticated()
        .exceptionHandling(exceptionHandling -> exceptionHandling.accessDeniedPage("/mauvaisrole.jsp"));

    return http.build();
}
```


Travaux pratiques

- Exo 19 - Importez le second exercice, ou réalisez vos propres pages
 - La page login devient la première page, la page menu ne doit pas être accédée si l'utilisateur n'est pas authentifié
 - La gestion des droits reste la même que précédemment
 - Une page login : accessible à tous
 - ❑ Si ok on part vers la page menu, si ko on repart vers la page login et on affiche un message
 - Réalisez une autre page qui s'affichera si l'utilisateur connecté n'a pas le bon rôle. Utilisez `<access-denied-handler>`
- Remarque : Testez dans un vrai navigateur, le navigateur interne d'Eclipse pouvant mal gérer le `<access-denied-handler>`



Spring Security – Logout

- Par défaut, on peut dire à spring security que l'on ne veut plus de la session en appelant le lien
 - En Spring Security 3 : `/j_spring_security_logout`
 - En Spring Security 4 : `/logout`
 - Cet URL est paramétrable via l'attribut **logout-url** de `<logout/>`
- Il fera un *invalidate* de session
- Vous devez l'activer et le paramétrer via la balise `<logout/>` qui se placera dans une balise `<http>`

Spring Security – Logout

- Tout comme `<form-login>` vous pouvez paramétrer le logout
 - Pour renvoyer vers une page spécifique par exemple via son attribut *logout-success-url*
 - Si vous ne souhaitez pas invalider la session, vous pouvez aussi faire usage de l'attribut *invalidate-session*
 - ❑ Il est à `true` par défaut

Spring Security – Logout

➤ Exemple de paramétrage code :

```
// Règles des matcher
// ...
// Pour tous les autres URL : il faut être authentifié
http.authorizeRequests().anyRequest().authenticated();

// Pour la page de logout
LogoutConfigurer<HttpSecurity> formLogout = http.authorizeRequests().and().logout();
// Toutes les pages associées à logout sont accessibles sans authentication
formLogout.permitAll();
formLogout.invalidateHttpSession(true);
formLogout.logoutUrl("/logout"); // <a href="/logout" ...>
formLogout.logoutSuccessUrl("/logoutok.jsp");
```

Spring Security – Logout

➤ Exemple de paramétrage code :

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

    http.authorizeHttpRequests(...)
        .logout(logout -> logout.logoutUrl("/logout")
            .logoutSuccessUrl("/logoutok.jsp")
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID")
            .permitAll()
        );

    return http.build();
}
```

6.1+

Travaux pratiques

- Exo 19 - Ajoutez une gestion Spring Security au lien dans la page menu afin de pouvoir se déconnecter
- Quand il se déconnecte, renvoyez l'utilisateur vers la page logoutok.jsp.



Spring Security – Remember-Me

- L'option remember-me permet au spring de reconnaître l'utilisateur
- On retrouve généralement cette option sous la forme d'une checkbox sur les sites webs

Créer mon compte Mon compte

Identifiez-vous

Pseudo

Mémoriser mon pseudo

Mot de passe

Date de naissance

JJ/MM/AAAA

Se connecter

Pseudo ou Mot de passe oublié ?

S'inscrire

Spring Security – Remember-Me

- Spring security sait gérer cette problématique
 - Il suffit d'ajouter une balise <remember-me> dans votre balise <http>
 - Dans votre formulaire de login, ajoutez une checkbox qui doit porter le *name*
 - ❑ En Spring Security 3 : **_spring_security_remember_me**
 - ❑ En Spring Security 4 : **remember_me**
 - ❑ Vous pouvez modifier cette option via l'attribut *remember-me-parameter*
- Cette option ne va pas réafficher le login/password, par défaut elle va conserver les informations de session dans un cookie sur le navigateur
 - Ainsi, **SI** l'utilisateur ne se délogue pas, il pourra accéder directement à une ressource **sans passer** par la page login
 - Le comportement peut varier en fonction des navigateurs ⇔ testez

Spring Security – Remember-Me

➤ Exemple de configuration code

```
// ...  
// Gestion du remember me  
RememberMeConfigurer<HttpSecurity> rememberMe = http.authorizeRequests().and().rememberMe();  
rememberMe.rememberMeParameter("rememberme");  
rememberMe.key("rememberme_key");  
rememberMe.rememberMeCookieName("rememberme_cookie");  
rememberMe.tokenValiditySeconds(600); // 10mn
```

```
<tr>  
  <td>Se souvenir de moi</td>  
  <td><input type="checkbox" name="rememberme" id="rememberme"/></td>  
</tr>
```

Formulaire Web

Spring Security – Remember-Me

➤ Exemple de configuration code

6.1+

// Gestion du remember me

```
.rememberMe(rememberMe -> rememberMe.rememberMeParameter("rememberme")  
    .key("rememberme_key")  
    .rememberMeCookieName("rememberme_cookie")  
    .tokenValiditySeconds(600)  
    .tokenRepository(this.persistentTokenRepository()));
```

```
<tr>  
  <td>Se souvenir de moi</td>  
  <td><input type="checkbox" name="rememberme" id="rememberme" /></td>  
</tr>
```

Formulaire Web

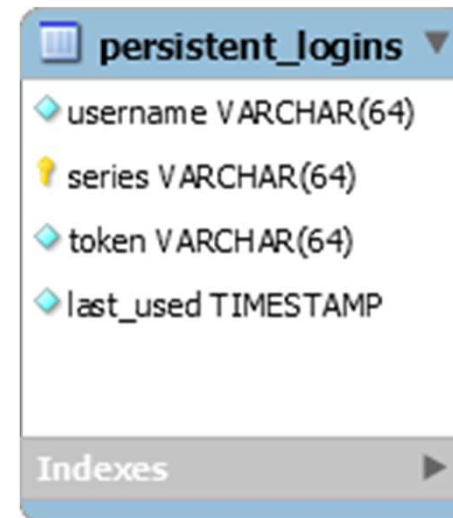
Spring Security – Remember-Me

- Attention : comme par défaut, tout est placé dans un cookie (login, pwd, expiration, ...), même crypté cela peut poser un problème de sécurité
 - ➡ Tout est **côté client**
- Vous pouvez compléter la configuration de remember-me via ses attributs et entre autres lui proposer de stocker les informations en base de données **côté serveur**
 - ➡ Attribut *data-source-ref* de la balise remember-me
 - ❑ Pointera vers un bean de type `DataSource`

Spring Security – Remember-Me

- Comme indiqué dans la documentation, cette data source devra pointer vers une base qui contient une table respectant les contraintes suivantes :

```
create table persistent_logins (  
  username varchar(64) not null,  
  series varchar(64) primary key,  
  token varchar(64) not null,  
  last_used timestamp not null)
```



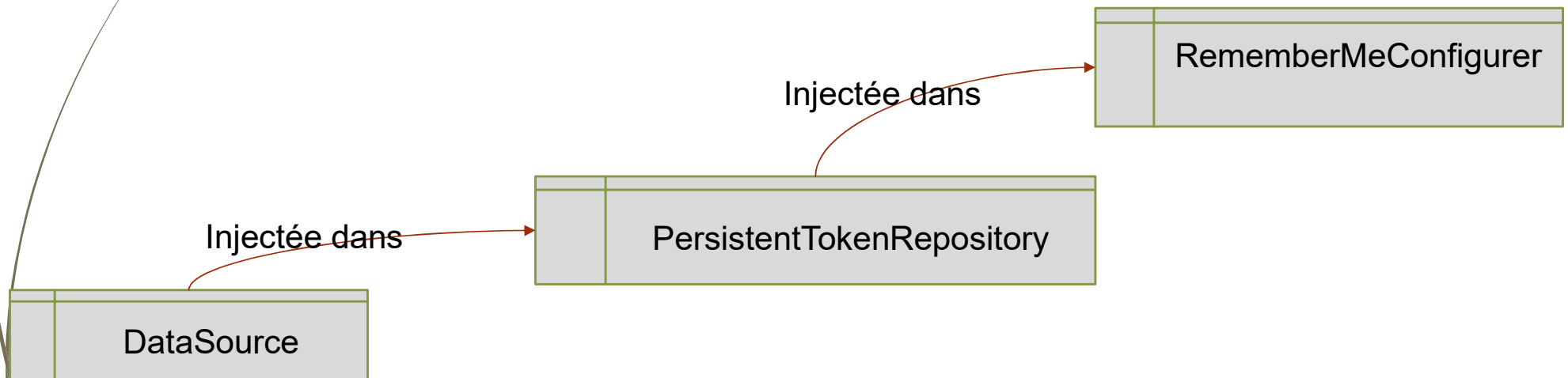
A screenshot of a database interface showing the structure of a table named 'persistent_logins'. The table has four columns: 'username' (VARCHAR(64)), 'series' (VARCHAR(64) marked as the primary key with a yellow key icon), 'token' (VARCHAR(64)), and 'last_used' (TIMESTAMP). Below the column list is a section labeled 'Indexes' with a right-pointing arrow.

persistent_logins	
username	VARCHAR(64)
series	VARCHAR(64)
token	VARCHAR(64)
last_used	TIMESTAMP

Indexes

Spring Security – Remember-Me

- Dans le cas de la configuration par classe annotée, il faudra créer un objet respectant l'interface **PersistentTokenRepository** et le lier à votre RememberMe



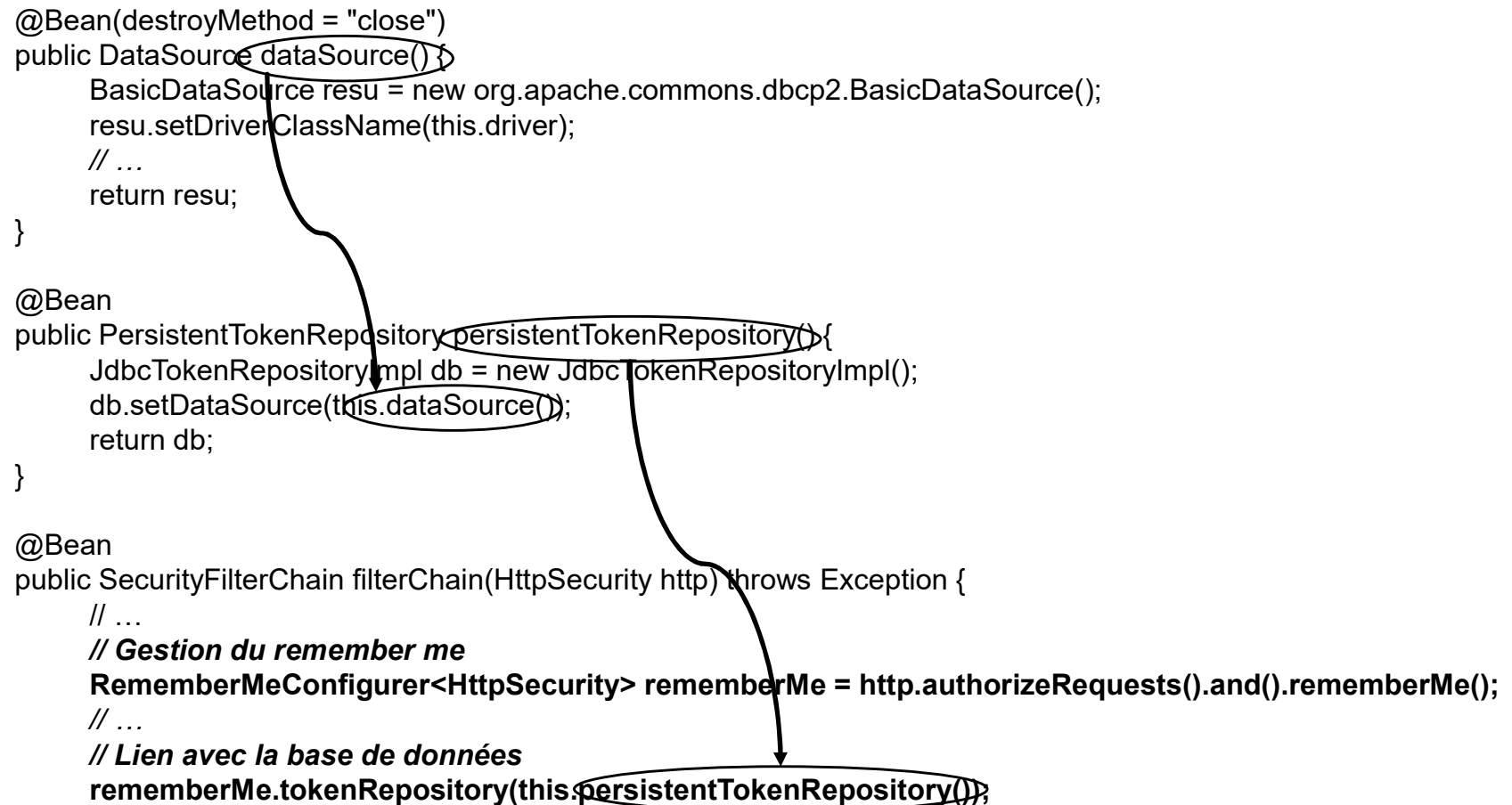
Spring Security – Remember-Me

➤ Exemple en code :

```
@Bean(destroyMethod = "close")
public DataSource dataSource() {
    BasicDataSource resu = new org.apache.commons.dbcp2.BasicDataSource();
    resu.setDriverClassName(this.driver);
    // ...
    return resu;
}

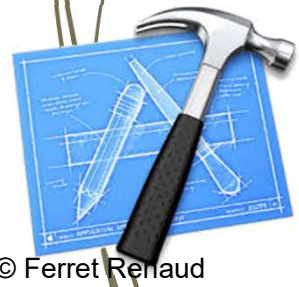
@Bean
public PersistentTokenRepository persistentTokenRepository() {
    JdbcTokenRepositoryImpl db = new JdbcTokenRepositoryImpl();
    db.setDataSource(this.dataSource());
    return db;
}

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    // ...
    // Gestion du remember me
    RememberMeConfigurer<HttpSecurity> rememberMe = http.authorizeRequests().and().rememberMe();
    // ...
    // Lien avec la base de données
    rememberMe.tokenRepository(this.persistentTokenRepository());
}
```



Travaux pratiques

- Exo 19 - Importer le troisième exercice spring security
 - Il reprend les éléments que l'on a codé
 - Il ajoute les dépendances vers le driver MySQL et commons-dbcp (ou commons-dbcp2)
- En utilisant le script qui est dans le dossier db, créez la table demandée par le spring security
- Déclarez une data source (faites exactement comme dans les exercices d'accès aux données)
- Branchez votre remember-me avec votre data source et allez sur votre site
 - Vérifiez dans la base de données que la table se remplit



Spring Security – URL

➤ Lors de vos déclarations vous avez filtré des URLs simples, vous pouvez faire usage des expressions suivantes :

➤ `/*` : tous les URLs à la racine du site

☐ `/toto.html` OK

☐ `/titi/toto.html` KO

➤ `/**` : tous les URLs à la racine du site et ses sous dossiers

☐ `/toto.html` OK

☐ `/titi/toto.html` OK

➤ `/comptes/**` : tous les URLs qui commence par `/comptes/`

☐ `/toto.html` KO

☐ `/comptes/toto.html` OK

➤ `/*.smvc` : tous les URLs qui sont à la racine et se terminent par `.smvc`

☐ `/toto.smvc` OK

☐ `/comptes/toto.smvc` KO

Spring Security – URL

- Vous pouvez ajouter à vos balises `<intercept-url>` des contraintes sur le protocole
 - C'est l'attribut *requires-channel* qui permet d'obtenir le filtrage désiré

```
<http>
  <intercept-url pattern="/secure/**" access="ROLE_USER" requires-channel="https"/>
  <intercept-url pattern="/**" access="ROLE_USER" requires-channel="any"/>
</http>
```

- Via le code il faut passer par *requiresChannel()*

```
http.authorizeRequests().antMatchers("/secure/**").access("hasRole('ROLE_USER')");
http.requiresChannel().antMatchers("/secure/**").requiresSecure();

http.authorizeRequests().antMatchers("/**").access("hasRole('ROLE_USER')");
```

Spring Security – Roles

- Les rôles commencent par **ROLE_** en majuscule
 - Ce n'est pas obligatoire, mais fortement conseillé
- C'est une simple chaîne de caractères
- Le rôle **ROLE_ANONYMOUS** existe, vous devez cependant l'activer en ajoutant la balise `<anonymous/>` dans votre `<http>`
 - Il peut être utilisé, par exemple, sur les URLs de ressources statiques

Spring Security – Roles

➤ Exemple en XML (sans les expressions)

```
<http>
  <anonymous />

  <intercept-url pattern="/login.jsp" access="ROLE_ANONYMOUS"/>
  <intercept-url pattern="/**/*.*js" access="ROLE_ANONYMOUS"/>
  <intercept-url pattern="/**" access="ROLE_USER"/>
</http>
```

➤ Exemple en XML (avec les expressions)

```
<http use-expressions="true">
  <anonymous />

  <intercept-url pattern="/login.jsp" access="isAnonymous()"/>
  <intercept-url pattern="/**/*.*js" access="hasRole('ROLE_USER')"/>
  <intercept-url pattern="/**" access="isAuthenticated()"/>
</http>
```

Spring Security – Roles

➤ Exemple en code

```
http.authorizeRequests().antMatchers("/adm/pageB.jsp*").access("hasRole('ROLE_ANONYMOUS')");  
http.authorizeRequests().antMatchers("/adm/pageC.jsp*").anonymous();
```

➤ En Spring Security 6+

```
http.authorizeHttpRequests(authorize ->  
    authorize.requestMatchers(new AntPathRequestMatcher("/user/pageA.jsp*")).hasRole("USER")  
        .requestMatchers(new AntPathRequestMatcher("/adm/pageC.jsp*")).anonymous()  
    );
```

6.1+

Spring Security – Authentication provider

- Authentication provider : a pour objectif de fournir des mécaniques d'authentifications
- Nous avons utilisé une liste en dur avec login/pwd/rôles
- Vous pouvez faire usage de mot de passes cryptés si vous le souhaitez, spring security sait gérer :

- plaintext
- sha
- md5
- md4
- ...

```
11< <authentication-provider>
12< <password-encoder hash="sha">
13< <salt-source system-wide="maClef" />
14< <!-- Ou si vous voulez une clef de hash par utilisateur -->
15< <!-- <salt-source user-property="username" /> -->
16< </password-encoder>
```

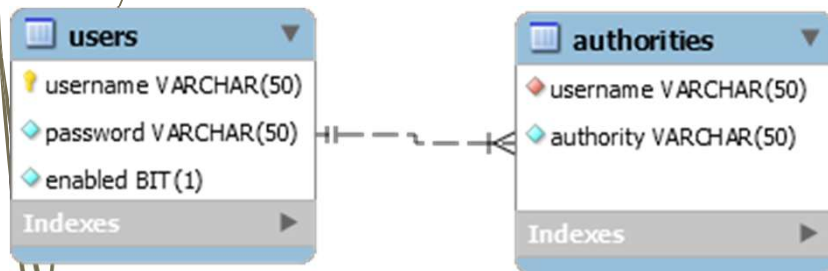
- Les cryptages nécessitant un *salt* ou *hash* devront se paramétrer dans le spring via la balise `<password-encoder>` dans `<authentication-provider>`

Spring Security – Authentication provider

- Nous pouvons aussi utiliser une base de données

```
9 <authentication-manager>
10 <authentication-provider>
11 <jdbc-user-service data-source-ref="securityDataSource"/>
12 </authentication-provider>
```

- Par défaut, cette base devra contenir deux tables



```
CREATE TABLE users (
    username VARCHAR(50) NOT NULL PRIMARY KEY,
    password VARCHAR(50) NOT NULL,
    enabled BIT NOT NULL
);
```

```
CREATE TABLE authorities (
    username VARCHAR(50) NOT NULL,
    authority VARCHAR(50) NOT NULL
);
```

```
ALTER TABLE authorities ADD CONSTRAINT fk_authorities_users foreign key
(username) REFERENCES users(username);
```

Spring Security – Authentication provider

- Sur une base de données existante, vous avez une certaine liberté
- Le paramétrage se fera au niveau `<jdbc-user-service>` (ou `JdbcUserDetailsManagerConfigurer`)
- Vous devrez simplement préciser certaines des requêtes SQL suivantes :
 - **usersByUsernameQuery** : doit récupérer (username, password, enabled) de votre base de données en fonction d'un username
 - **authoritiesByUsernameQuery** : doit récupérer (username, authority) de votre base de données en fonction d'un username
 - **groupAuthoritiesByUsername** : doit récupérer (id, group_name, authority) de votre base de données en fonction d'un username
- Vous devrez peut être faire usage d'alias si vous partez d'une base existante.

Spring Security – Authentication provider

- Si vous avez déjà une base de données contenant une table utilisateur et une table rôle vous pouvez paramétrer votre `<jdbc-user-service>` afin d'en tenir compte
- Dans le cas d'un LDAP, la configuration ne se fera pas via l'authentication-provider
 - <https://docs.spring.io/spring-security/site/docs/current/reference/html5/#ldap>
- Dans les cas plus complexes, vous devrez réaliser votre propre provider

Spring Security – Authentication provider

➤ Votre Authentication Provider

```
@Service
public class AuthenticationService implements AuthenticationProvider {
    @Autowired
    private IUtilisateurDAO utilisateurDAO;

    @Override
    @Transactional(readOnly = true, rollbackFor = Exception.class)
    public Authentication authenticate(Authentication authentication) throws AuthenticationException {
        String name = authentication.getName();
        String password = authentication.getCredentials() != null ? authentication.getCredentials().toString() : null;
        ...
        GrantedAuthority ga = new SimpleGrantedAuthority("ROLE_USER");
        Collection<GrantedAuthority> roles = new ArrayList<>(1);
        roles.add(ga);
        return new UsernamePasswordAuthenticationToken(name, password, roles);
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return Authentication.class.isAssignableFrom(authentication);
    }
}
```

Spring Security – Authentication provider

➤ Branchement dans Spring Security de votre Authentication Provider & Manager

Ancienne
approche

```
@Configuration
@EnableWebSecurity
public class SpringSecurityConfiguration extends WebSecurityConfigurerAdapter {

    @Autowired
    private AuthenticationProvider customAuthenticationProvider;

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws Exception {
        // On branche simplement notre provider
        auth.authenticationProvider(this.customAuthenticationProvider);
    }
    ...
}
```

Spring Security – Authentication provider

➤ Branchement dans Spring Security de votre Authentication Provider & Manager

Approche
actuelle

```
@Configuration
@EnableWebSecurity
public class SpringSecurityConfiguration {

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration authenticationConfiguration)
        throws Exception {
        return authenticationConfiguration.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        ...
        return http.build();
    }
}
```

Spring Security – Dans ses pages JSP

- Vous pouvez faire usage d'une tag lib de spring security dans vos page JSP
 - N'oubliez pas la dépendance Maven spring-security-taglibs
- Son URI est :

```
<%@ taglib uri="http://www.springframework.org/security/tags" prefix="sec" %>
```

Spring Security – Dans ses pages JSP

➤ Elle vous permettra par exemple

- De récupérer un attribut de la classe Authentication du Spring security

```
<sec:authentication property="principal.username"/>
```

- De tester si un utilisateur a le/les bons rôles

```
<sec:authorize access="hasRole('ROLE_USER')">...</sec:authorize>  
<sec:authorize access="hasRole('ROLE_ADMIN')">...</sec:authorize>  
<sec:authorize access="isFullyAuthenticated()">...</sec:authorize>  
<sec:authorize access="hasRole('ROLE_ADMIN') and !hasRole('ROLE_USER')">...</sec:authorize>
```

Spring Security – Dans ses pages JSP

- Attention : pour que `<sec:authorize access="xxx">` fonctionne, vous devez **impérativement**
 - Ajouter dans votre balise `<http>` l'option **`use-expressions="true"`**
 - Et définir vos rôles en faisant usage des expressions :

```
<intercept-url pattern="/adm/pageB.jsp" access="hasRole('ROLE_ADMIN')" />
```

Spring Security – Expression

➤ Voici quelques expressions utilisables :

- `hasRole([role])` Returns true if the current principal has the specified role.
- `hasAnyRole([role1,role2])` Returns true if the current principal has any of the supplied roles (given as a comma-separated list of strings)
- `principal` Allows direct access to the principal object representing the current user
- `authentication` Allows direct access to the current Authentication object obtained from the SecurityContext
- `permitAll` Always evaluates to true
- `denyAll` Always evaluates to false
- `isAnonymous()` Returns true if the current principal is an anonymous user
- `isRememberMe()` Returns true if the current principal is a remember-me user
- `isAuthenticated()` Returns true if the user is not anonymous
- `isFullyAuthenticated()` Returns true if the user is not an anonymous or a remember-me user
- `hasIpAddress('192.168.2.1')` Ip filtering

Spring Security – Expression

- Vous pouvez aussi utiliser les opérateurs de Spring EL
 - and, or, ...

```
hasRole('admin') and hasIpAddress('192.168.1.0/24')
```

- Vous pouvez appeler vos propres méthodes de vérification

```
@webSecurity.check(authentication,request)
```

- Vous pouvez faire usage de paramètres

```
@webSecurity.checkUserId(authentication,#userId)
```

- <https://docs.spring.io/spring-security/site/docs/current/reference/html5/#el-access>

Spring Security – Dans le code Java

- Côté `@Controller`, `@Service`, `@Repository` ... vous pouvez ajouter une annotation spring security qui se chargera de vérifier que la personne qui veut appeler votre code a le/les bons rôles.

- **@Secured** : ne sait pas utiliser les expressions

- **@PreAuthorize** : sait utiliser les expressions et se fait avant l'appel à la méthode

- **@PostAuthorize** : sait utiliser les expressions et se fait après l'appel à la méthode

- Toutes ses annotations se placent sur les méthodes

Préférez
son usage à
`@Secured`

Spring Security – Dans le code Java

- Pour en faire usage, vous devez l'indiquer dans votre fichier spring security

```
<global-method-security pre-post-annotations="enabled" secured-annotations="enabled" />
```

- Dans le cas des classes de configuration utilisez **@EnableGlobalMethodSecurity**

```
@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true, securedEnabled = true)
public class MaConfigurationSpringSecurity {
    // Avec ou sans héritage de WebSecurityConfigurerAdapter
    // ...
}
```

- Remarque : La gestion de la sécurité au niveau du code se gère de préférence sur les **@Service**
 - **Pas** dans les **@Controller** ou **@Repository**

Spring Security 6 – Dans le code Java

- Si vous êtes en **Spring Security 6** l'annotation de classe `@EnableGlobalMethodSecurity` est remplacée par **`@EnableMethodSecurity`**

```
@Configuration
@EnableWebSecurity // Uniquement si vous n'êtes pas en Spring Boot
@EnableMethodSecurity(prePostEnabled = true, securedEnabled = true)
public class MaConfigurationSpringSecurity6 {
    // ...
}
```

Travaux pratiques



- Exo 19 - Vérifiez que vous avez bien la dépendance nécessaire pour faire usage de la tag lib spring security
- Si besoin, modifiez votre fichier spring security afin de faire usage des expressions
- Modifiez la page menu afin
 - de cacher les URLs en fonction des rôles (pageA, pageB, logout)
 - De dire bonjour à la personne qui s'est connectée sur la page menu
 - Ne modifiez pas les pageA et pageB
- Dans le code Java de votre service métier, faites usage des annotations pour indiquer que la méthode
 - faireAdmin n'est utilisable que par ceux qui ont un rôle admin
 - faireUser n'est utilisable que par ceux qui ont un rôle user

Spring Security – Context

- De partout dans votre code vous pouvez récupérer le context de sécurité Spring

```
SecurityContextHolder.getContext();
```

- Avec cette approche, les informations qui se trouvent dans le token Spring Security se retrouvent accessibles d'où vous voulez

- ➡ Privilégiez l'usage des annotations **@PreAuthorize**

```
Authentication principal = SecurityContextHolder.getContext().getAuthentication();
```

Spring Security – @AuthenticationPrincipal

- Si vous voulez récupérer l'objet représentant l'authentification (*Authentication.getPrincipal()*), vous pouvez aussi vous le faire injecté via l'annotation **@AuthenticationPrincipal**

```
@RestController
public class SocialApplication {

    @GetMapping("/user")
    public Object user(@AuthenticationPrincipal Object principal) {
        return principal;
    }
}
```

```
@RestController
public class SimpleController {
    @GetMapping("/whoami")
    public String whoami(
        @AuthenticationPrincipal(expression="name") String name) {
        return name;
    }
}
```

Travaux pratiques

➤ Exo 19 - Pour aller plus loin

- Intégrer le spring security dans l'exercice banque qui fait usage du Spring MVC en mode JSP / Servlet.
- Deux options
 - ☐ Compléter la base et ajouter deux trois éléments
 - ☐ Ajuster le code pour le rendre compatible avec Spring Security



Spring Security – WS

- Dans le cas des WS nous n'aurons pas de page WEB pour login et logout
- Nous ferons usage du JsonWebToken par exemple
- Dans ce cas, tout sera géré par deux Filter
 - ➡ Le premier se chargera de l'authentification et de la création du token JWT
 - ➡ Le second sera responsable de l'interception du token

Spring Security – WS

- La gestion des erreurs
 - Sur l'authentification
 - Sur l'habilitation

- Se fera via respectivement
 - Un `authenticationEntryPoint`
 - Un `accessDeniedHandler`

Spring Security – WS et JWT

74

```
@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration authenticationConfiguration) throws Exception {
    return authenticationConfiguration.getAuthenticationManager();
}

@Bean
public SecurityFilterChain filterChain(HttpSecurity http, AuthenticationManager authenticationManager) throws Exception {
    // On desactive CSRF et on active la configuration CORS
    http.csrf().disable().cors();
    // Nos règles
    http.authorizeRequests().antMatchers("/", "/favicon.ico").anonymous();

    // Autres contraintes ...

    // La gestion du JWT
    // 2 Filters :
    // - JwtAuthenticationFilter : UsernamePasswordAuthenticationFilter gère l'authentification et la première fabrication du token
    // - JwtAuthorizationFilter : BasicAuthenticationFilter gère la récupération du token par la suite
    http.authorizeRequests().and()
        .addFilterBefore(
            new JwtAuthenticationFilter(authenticationManager, this.env), UsernamePasswordAuthenticationFilter.class)
        .addFilterBefore(
            new JwtAuthorizationFilter(authenticationManager, this.env), UsernamePasswordAuthenticationFilter.class)
        .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS);
}
```

Spring Security – WS et JWT

➤ Exemple (suite)

```
// Gestion des erreurs en cas de problème
// Dedans, on gère le HTTP status et le flux JSON
ExceptionHandlerConfigurer<HttpSecurity> exceptionHandling = http.authorizeRequests().and().exceptionHandling();
exceptionHandling.accessDeniedHandler(new AccesDeniedHandler());
// Attention : peut générer des effets de bord avec des systèmes comme OpenID
exceptionHandling.authenticationEntryPoint(new RestAuthenticationEntryPoint());

// Le logout
http.authorizeRequests().and().logout().clearAuthentication(true)
    .logoutSuccessHandler((pRequest, pResponse, pAuthentication) -> pResponse.setStatus(200));

// Pas de login gere par Spring Security classique
http.authorizeRequests().and().formLogin().disable().httpBasic().disable();

return http.build();
} // Fin de la méthode public SecurityFilterChain filterChain(HttpSecurity, AuthenticationManager)
```

Spring Security – WS

- **UsernamePasswordAuthenticationFilter** : Filtre qui va s'occuper de l'authentification et de la fabrication du JWT

```
public class JwtAuthenticationFilter extends UsernamePasswordAuthenticationFilter {  
  
    // Seront injectes automatiquement via le constructeur  
    private final Environment env;  
    private final AuthenticationManager authenticationManager;  
  
    public JwtAuthenticationFilter(AuthenticationManager pAuthenticationManager, Environment pEnv) {  
        this.authenticationManager = pAuthenticationManager;  
        // Ici on indique l'URL du login  
        this.setFilterProcessesUrl("/login");  
        this.env = pEnv;  
    }  
}
```

Spring Security – WS

➤ UsernamePasswordAuthenticationFilter : suite

```
@Override
```

```
public Authentication attemptAuthentication(HttpServletRequest request, HttpServletResponse response) {  
    // Ici, on se débrouille pour récupérer les éléments nécessaires à l'authentification  
    // Par exemple, en les récupérant du body ou de la request  
  
    // On va aussi fabriquer le Token Spring Security  
    Authentication authenticationToken = new UsernamePasswordAuthenticationToken(login, password);  
  
    // Et appeler le service d'authentification afin de retourner le résultat  
    return this.authenticationManager.authenticate(authenticationToken);  
}
```

Spring Security – WS

➤ UsernamePasswordAuthenticationFilter : suite

```
@Override
protected void successfulAuthentication(HttpServletRequest request,
                                       HttpServletResponse response,
                                       FilterChain filterChain, Authentication authentication) {

    // On va construire dans cette méthode le JWT
    Claims claims = new DefaultClaims();
    // On peut y placer toutes les informations que l'on souhaite

    // Puis on l'encode, par exemple ici en HS512
    byte[] signingKey = this.env.getProperty("configuration.jwt.key", "xxxxxyyyy").getBytes();

    JwtBuilder builder = Jwts.builder().signWith(Keys.hmacShaKeyFor(signingKey),
                                                SignatureAlgorithm.HS512).setHeaderParam("typ", "JWT").setClaims(claims);

    String token = builder.compact();
    // Puis on le place dans le Header par exemple
    response.addHeader("Authorization", "Bearer " + token);
}
```

Spring Security – WS

➤ **BasicAuthenticationFilter** : Filtre qui va s'occuper de valider le JWT et de le réinjecter dans Spring Security

```
public class JwtAuthorizationFilter extends BasicAuthenticationFilter {  
    // Sera injecte via le constructeur  
    private final Environment env;  
  
    public JwtAuthorizationFilter(AuthenticationManager pAuthenticationManager, Environment pEnv) {  
        super(pAuthenticationManager);  
        this.env = pEnv;  
    }  
}
```

Spring Security – WS

➤ **BasicAuthenticationFilter** : suite

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
    throws IOException, ServletException {

    // On récupère le JWT (du header par exemple)
    String jwtToken = request.getHeader("Authorization");

    // Si il est vide on ne fait rien (un log à la rigueur)
    // Sinon
    // - on le decrypte
    // - on fabrique un Token Spring Security (ici authentication)
    // - On l'injecte dans le Spring Security Context
    SecurityContextHolder.getContext().setAuthentication(authentication);
    filterChain.doFilter(request, response);
}
```


Spring Security – WS

- **AuthenticationEntryPoint** : objet qui va générer un code d'erreur HTTP (401) quand l'utilisateur n'est pas authentifié
 - Il appartient au client appelant de partir vers une page si il récupère un code 401
 - Attention : cet entry point peut poser des problèmes avec l'OpenID

```
public final class RestAuthenticationEntryPoint implements AuthenticationEntryPoint {  
  
    @Override  
    public void commence(  
        HttpServletRequest request,  
        HttpServletResponse response,  
        AuthenticationException authException) throws IOException, ServletException {  
  
        response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized"); // 401  
    }  
}
```

Spring Security – WS

- **AccessDeniedHandler** : objet qui va générer un code d'erreur HTTP (403) quand l'utilisateur n'a pas le bon rôle

```
public final class AccessDeniedHandler implements AccessDeniedHandler {  
  
    @Override  
    public void handle(HttpServletRequest request,  
                      HttpServletResponse response, AccessDeniedException exception)  
        throws IOException, ServletException {  
        response.sendError(HttpServletResponse.SC_FORBIDDEN, "Forbidden");  
    }  
}
```

Spring Security – CrossOrigin

- La gestion du CrossOrigin pour se globaliser via la déclaration d'un Bean dédié de type **CorsConfigurationSource**
 - Il est préférable de le déclarer dans votre classe/fichier dédié à Spring Security
- **Attention** : vous devrez aussi activer ce bean via le code : `http.cors();` dans votre méthode *configure* (ou *filterChain*)

Spring Security – CrossOrigin

➤ Exemple

```
@Bean
protected CorsConfigurationSource corsConfigurationSource() {
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowCredentials(true);
    // Se transformera en addAllowedOriginPattern en Spring 5.3+
    config.addAllowedOrigin("*");

    config.addAllowedHeader("*");
    config.addAllowedMethod("*");

    config.addExposedHeader("WWW-Authenticate");
    config.addExposedHeader("Access-Control-Allow-Origin");
    config.addExposedHeader("Access-Control-Allow-Headers");
    // Si, on veut voir notre token JWT côté JS pour Angular par exemple
    config.addExposedHeader("Authorization");

    source.registerCorsConfiguration("/*", config);
    return source;
}
```

Spring Security – CrossOrigin

➤ Ne surtout pas oublier de l'activer

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http, AuthenticationManager authenticationManager)
    throws Exception {
    // Ici, pour notre exemple :
    // - on désactive le CSRF
    // - on active le CORS
    http.csrf().disable().cors();
    ...
    return http.build();
}
```

➤ CSRF : Cross-Site Request Forgery

➡ <https://www.cert.ssi.gouv.fr/information/CERTA-2008-INF-003/>

Travaux pratiques

- Exo 19 - P7 - Pour aller plus loin
 - Intégrer le spring security dans l'exercice banque qui fait usage du Spring MVC en mode REST



Spring Security – OpenID

- Idée : demander de s'authentifier à un tier
 - ➡ Par exemple Google, GitHub, Facebook ...

- Toute la mécanique
 - ➡ d'authentification sera gérée par le tier
 - ❑ C'est lui qui décide des identifiants / mots de passes
 - ➡ de token sera géré par Spring Security
 - ❑ Le token étant fabriqué par le tier

Spring Security – OpenID

➤ Pour en faire usage :

➤ Dans son pom.xml :

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-oauth2-client</artifactId>  
</dependency>
```

➤ Dans son fichier application.properties

❑ Il faudra fournir

```
spring.security.oauth2.client.registration.xxx.clientId=zzzz  
spring.security.oauth2.client.registration.xxx.clientSecret=yyyy
```

○ xxx : le nom du tier

Spring Security – OpenID

- Pour obtenir le clientId et le clientSecret, il faudra suivre les contraintes associées à chaque tier
 - GitHub : <https://github.com/settings/developers>
 - Google :
<https://developers.google.com/identity/protocols/oauth2/openid-connect#appsetup>
 - ...
- Vous pouvez en placer plusieurs tiers dans le fichier de configuration

Spring Security – OpenID

- La configuration Spring Security se simplifie
 - Plus besoin de gérer
 - ❑ les mots de passes
 - ❑ les URLs de login / logout (géré par le tier)
 - ❑ les token
 - Il faut simplement lui indiquer que vous êtes en OAuth2
 - ❑ via la méthode `oauth2Login()`

Spring Security – OpenID

➤ Exemple :

```
// Keep cors enable here, otherwise configuration of it is not applied
http.csrf().disable().cors();

http.authorizeRequests().antMatchers("/", // Root
    "/favicon.ico*").permitAll();

// For logout, simply send 200
http.authorizeRequests().and().logout().clearAuthentication(true)
    .logoutSuccessHandler((pRequest, pResponse, pAuthentication) -> pResponse.setStatus(200));

// No login, and no logout
http.authorizeRequests().and().formLogin().disable().httpBasic().disable();

// Other constraint are handled at the controller level with annotations
http.authorizeRequests().anyRequest().authenticated();

// Simple
http.authorizeRequests().anyRequest().authenticated().and().oauth2Login();
```

Spring Security – OpenID

- Ce sera à vous de faire le lien entre le user authentifié par openId et votre base de données

```
public UtilisateurEntity getUtilisateurConnecte() {  
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();  
    if (authentication.getPrincipal() instanceof OidcUser) {  
        OidcUser principal = (OidcUser) authentication.getPrincipal();  
        return this.getUtilisateurDAO().findByLogin(principal.getEmail()).get();  
    }  
    return null;  
}
```

Travaux pratiques

- Exo 19 - P8 - Pour aller plus loin
 - Intégrer le open-id avec Google dans l'application Rest avec la banque

